

Annotated
Version

Machine Learning Course - CS-433

Optimization

Sep 11+17, 2024

Martin Jaggi

Last updated on: September 17, 2024

credits to Mohammad Emtiyaz Khan

EPFL

Learning / Estimation / Fitting

Given a cost function $\mathcal{L}(\mathbf{w})$, we wish to find $\mathbf{w}^*$ which minimizes the cost:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad \text{subject to } \mathbf{w} \in \mathbb{R}^D$$

This means the *learning* problem is formulated as an **optimization problem**.

We will use an **optimization algorithm** to solve the problem (to find a good $\mathbf{w}$).

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\mathbf{w})$$

if

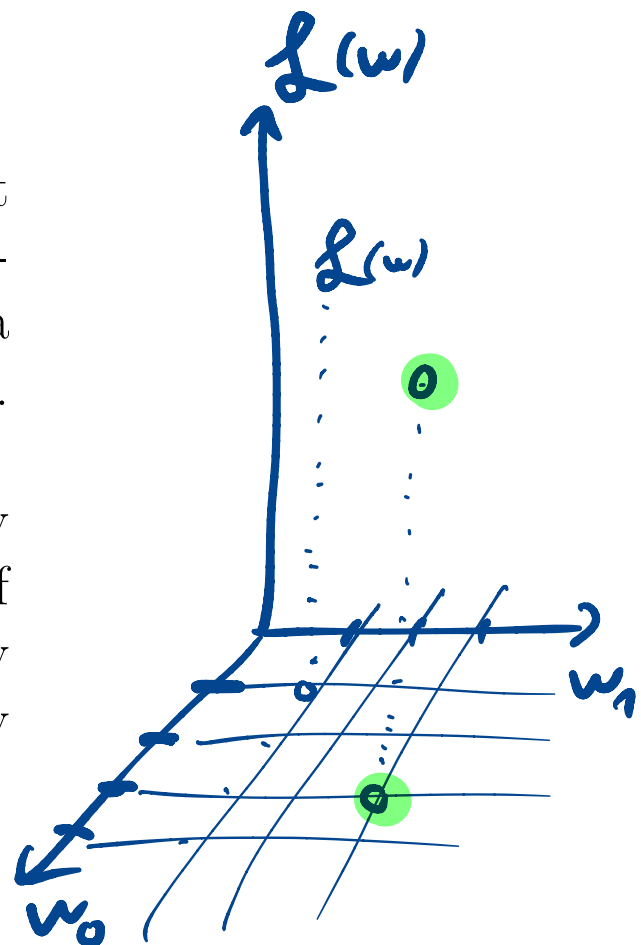
$$\text{loss}(y_n - f_{\mathbf{w}}(x_n))$$

↑
model

Grid Search

Grid search is one of the simplest optimization algorithms. We compute the cost over all values $\mathbf{w}$ in a grid, and pick the best among those.

This is brute-force, but extremely simple and works for any kind of cost function when we have very few parameters and the cost is easy to compute.



$$D=2$$

For a large number of parameters D , however, grid search has too many “for-loops”, resulting in an exponential computational complexity:

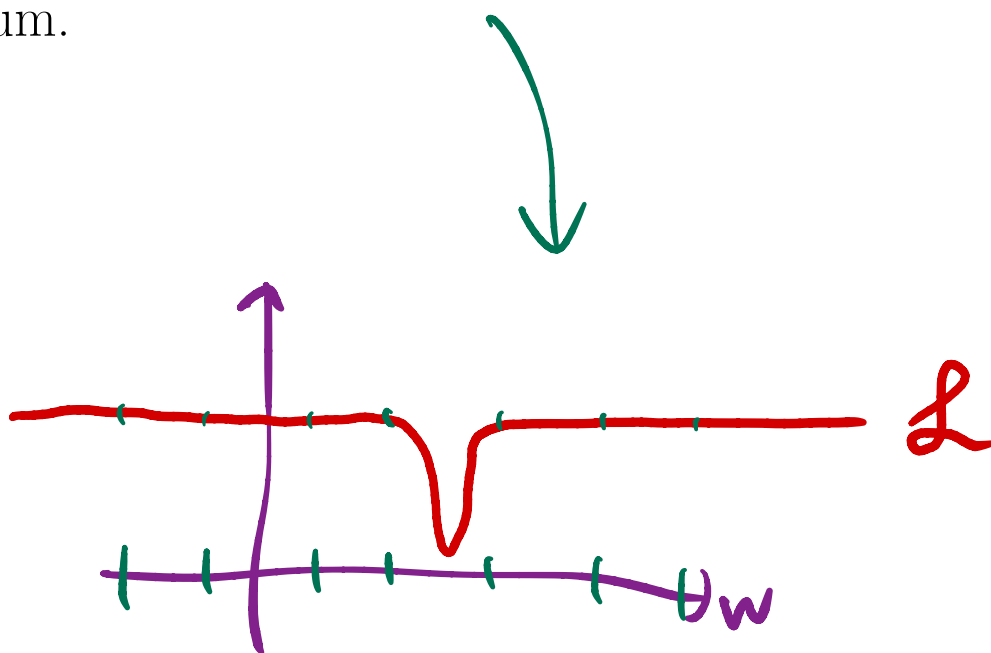
If we decide to use 10 possible values for each dimension of $\mathbf{w}$, then we have to check 10^D points.

This is clearly impossible for most practical machine learning models, which can often have $D \approx$ millions of parameters. Choosing a good range of values for each dimension is another problem.

Other issues: No guarantee can be given that we end up close to an optimum.

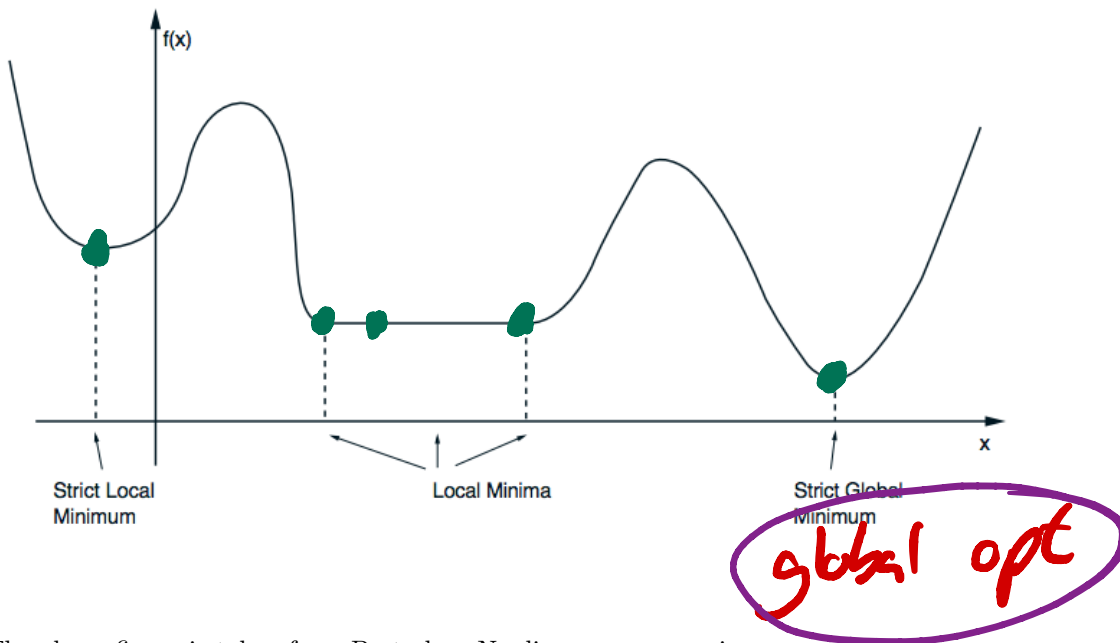
exponential
in dimension D
||
param

$10^D \dots$



granularity of search

Optimization Landscapes



The above figure is taken from Bertsekas, Nonlinear programming.

A vector $\mathbf{w}^*$ is a local minimum of $\mathcal{L}$ if it is no worse than its neighbors; i.e. there exists an $\epsilon > 0$ such that,

$$\mathcal{L}(\mathbf{w}^*) \leq \mathcal{L}(\mathbf{w}), \quad \forall \mathbf{w} \text{ with } \|\mathbf{w} - \mathbf{w}^*\| < \epsilon$$

A vector $\mathbf{w}^*$ is a global minimum of $\mathcal{L}$ if it is no worse than all others,

$$\mathcal{L}(\mathbf{w}^*) \leq \mathcal{L}(\mathbf{w}), \quad \forall \mathbf{w} \in \mathbb{R}^D$$

A local or global minimum is said to be strict if the corresponding inequality is strict for $\mathbf{w} \neq \mathbf{w}^*$.

Smooth Optimization

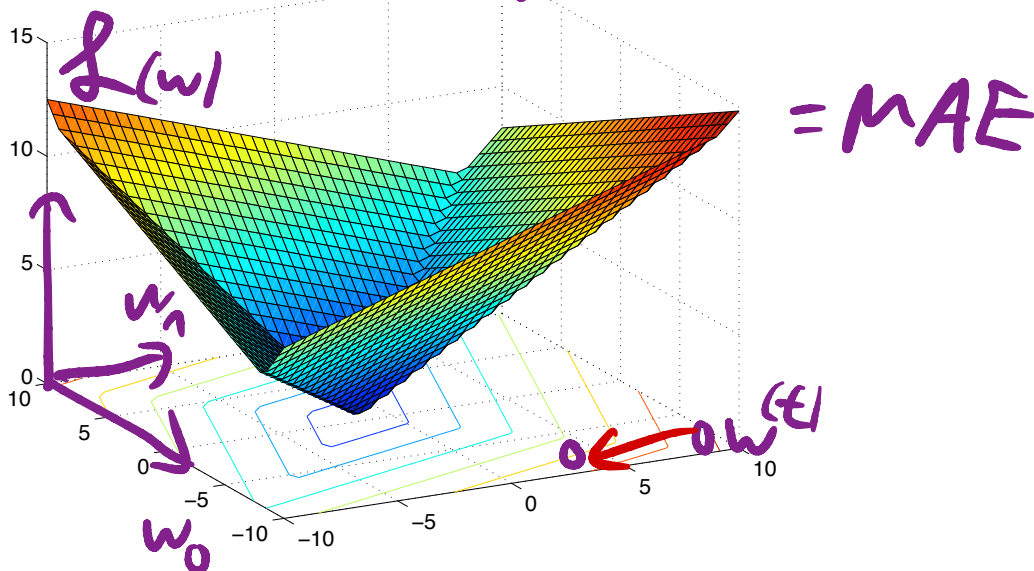
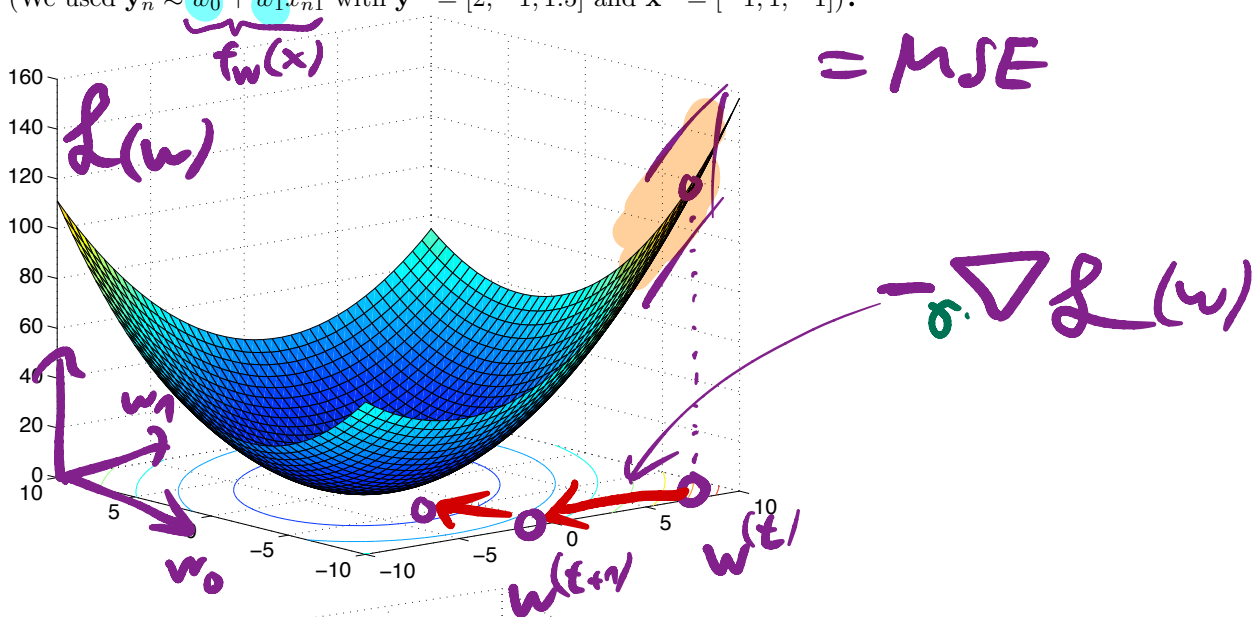
Follow the Gradient

A gradient (at a point) is the slope of the tangent to the function (at that point). It points to the direction of largest increase of the function.

Cauchy
1848

For a 2-parameter model, MSE($\mathbf{w}$) and MAE($\mathbf{w}$) are shown below.

(We used $\mathbf{y}_n \approx w_0 + w_1 x_{n1}$ with $\mathbf{y}^\top = [2, -1, 1.5]$ and $\mathbf{x}^\top = [-1, 1, -1]$).



Definition of the gradient:

$$\nabla \mathcal{L}(\mathbf{w}) := \left[\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial \mathcal{L}(\mathbf{w})}{\partial w_D} \right]^\top$$

This is a vector, $\nabla \mathcal{L}(\mathbf{w}) \in \mathbb{R}^D$.

Gradient Descent

To minimize the function, we iteratively take a step in the (opposite) direction of the gradient

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}(\mathbf{w}^{(t)})$$

where $\gamma > 0$ is the **step-size** (or **learning rate**). Then repeat with the next t .

Example: Gradient descent for 1-parameter model to minimize MSE:

$$w_0^{(t+1)} := (1 - \gamma)w_0^{(t)} + \gamma \bar{y}$$

where $\bar{y} := \sum_n y_n / N$. When is this sequence guaranteed to converge?

popular step-size
 $\gamma = \frac{\text{const}}{t}$

diverge
 $\downarrow +\infty$

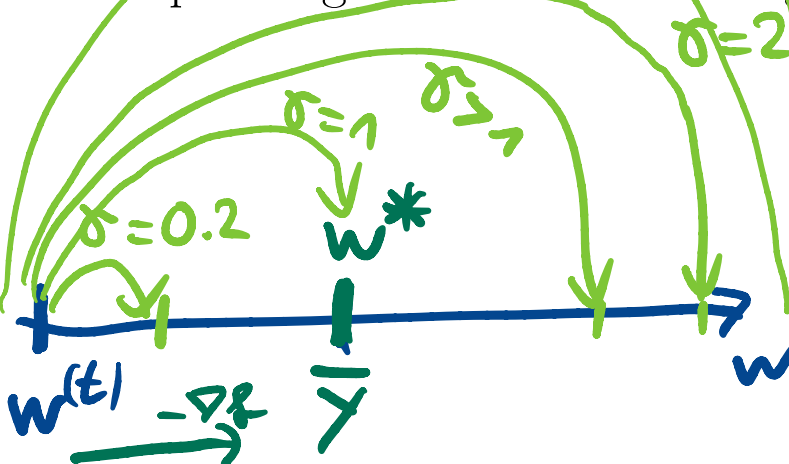
$\min_w \mathcal{L}(w)$

1-param $w = w_0 = f_w(x)$

$$\mathcal{L}(w) = \frac{1}{2N} \sum_{n=1}^N (y_n - w_0)^2$$

$$\nabla \mathcal{L} = \frac{\partial}{\partial w_0} \mathcal{L}$$

$$\begin{aligned} &= \frac{1}{2N} \sum_{n=1}^N -2(y_n - w_0) \\ &= \left(-\frac{1}{N} \sum_{n=1}^N y_n \right) + w_0 \\ &= w_0 - \bar{y} \end{aligned}$$



$$\mathbf{x}_n \in \mathbb{R}^D$$

$$\mathbf{w} \in \mathbb{R}^D$$

Gradient Descent for Linear MSE

For linear regression

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix}$$

Diagram: A purple bracket labeled 'D' spans the columns of matrix X. A purple bracket labeled 'N' spans the rows of matrix X. An orange horizontal bar highlights the row x_n in matrix X, with an arrow pointing to it from the label x_n . A green vertical bar highlights the column x_{n2} in matrix X.

We define the error vector $\mathbf{e}$:

$$\mathbf{e} = \mathbf{y} - \mathbf{X}\mathbf{w} \in \mathbb{R}^N$$

and MSE as follows:

$$\mathcal{L}(\mathbf{w}) := \frac{1}{2N} \sum_{n=1}^N \underbrace{\left(y_n - \mathbf{x}_n^\top \mathbf{w} \right)^2}_{\mathbf{e}_n^2}$$

Diagram: A red bracket labeled $f_w(\mathbf{x})$ is above the term $y_n - \mathbf{x}_n^\top \mathbf{w}$. A green bracket labeled $\mathbf{e}_n$ is below the term $y_n - \mathbf{x}_n^\top \mathbf{w}$. A red bracket labeled $\mathcal{L}_n(\mathbf{w})$ is below the entire sum.

then the gradient is given by

$$\nabla \mathcal{L}(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^\top \mathbf{e}$$

Diagram: A red vector $\mathbf{x}_n^\top$ (row vector) is shown next to a red vector $\mathbf{w}$ (column vector).

$$\begin{aligned} \frac{\partial}{\partial w_1} \mathcal{L} &= \frac{1}{2N} \sum_{n=1}^N -2(y_n - \mathbf{x}_n^\top \mathbf{w}) \cdot x_{n1} \\ &= -\frac{1}{N} (\mathbf{x}_{:1})^\top \mathbf{e} \\ &\vdots \\ \frac{\partial}{\partial w_D} \mathcal{L} &= \dots \sum -(\dots) \cdot x_{nD} \\ &= -\frac{1}{N} (\mathbf{x}_{:D})^\top \mathbf{e} \end{aligned}$$

Computational cost. What is the complexity (# operations) of computing the gradient?

a) starting from $\mathbf{w}$ and

b) given $\mathbf{e}$ and $\mathbf{w}$?

① compute $\mathbf{e}$

cost $\mathcal{O}(N \cdot D)$

② compute $\mathbf{X}^\top \mathbf{e} = \nabla \mathcal{L}$

cost $\mathcal{O}(D \cdot N)$

+ $\mathcal{O}(N \cdot D)$

Variant with offset. Recall: Alternative trick when also incorporating an offset term for the regression:

$$\mathbf{w} \in \mathbb{R}^{D+1}$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad \tilde{\mathbf{X}} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1D} \\ 1 & x_{21} & x_{22} & \dots & x_{2D} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & x_{N2} & \dots & x_{ND} \end{bmatrix}$$

Stochastic Gradient Descent

SGD

Sum Objectives. In machine learning, most cost functions are formulated as a **sum** over the training examples, that is

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\mathbf{w}) ,$$

where $\mathcal{L}_n$ is the cost contributed by the n -th training example.

$$\mathcal{L}_n = \text{loss}(y_n - f_{\mathbf{w}}(x_n))$$

Q: What are the $\mathcal{L}_n$ for linear MSE?

The SGD Algorithm. The **stochastic gradient descent** (SGD) algorithm is given by the following update rule, at step t :

stochastic gradient

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}_n(\mathbf{w}^{(t)}) .$$

①

$n \in \{1, \dots, N\}$ random

Theoretical Motivation. *Idea:* Cheap but unbiased **estimate** of the gradient!

In expectation over the random choice of n , we have

$$\mathbb{E} [\overset{=g}{\nabla \mathcal{L}_n(\mathbf{w})}] = \boxed{\nabla \mathcal{L}(\mathbf{w})}$$

which is the true gradient direction.
(check!)

Mini-batch SGD. There is an intermediate version, using the update direction being

$$\mathbf{g} := \frac{1}{|B|} \sum_{n \in B} \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$$

again with

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g}.$$

In the above gradient computation, we have randomly chosen a subset $B \subseteq [N]$ of the training examples. For each of these selected examples n , we compute the respective gradient $\nabla \mathcal{L}_n$, at the same current point $\mathbf{w}^{(t)}$.

$$\begin{aligned} & \mathbb{E}_n [\nabla \mathcal{L}_n(\mathbf{w})] \\ &= \sum_{n=1}^N \frac{1}{N} \nabla \mathcal{L}_n(\mathbf{w}) \\ &= \nabla \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(\mathbf{w}) \\ &= \nabla \mathcal{L}(\mathbf{w}) \end{aligned}$$

$$B \subseteq \{1, \dots, N\}$$

$|B|$ sample size

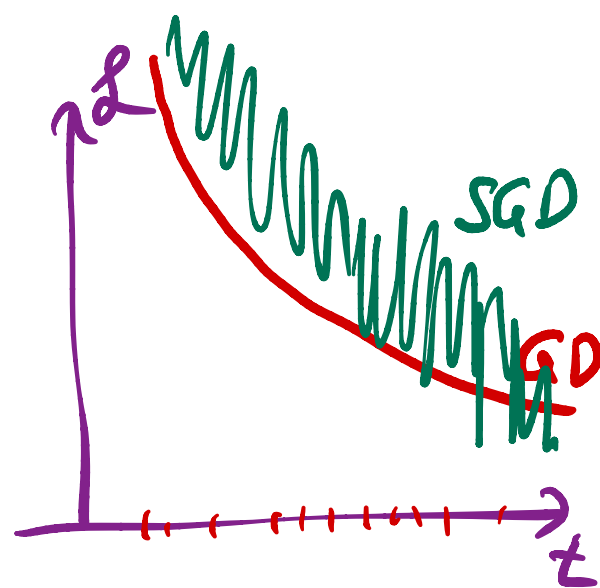
Extreme cases:

$B=1 \rightarrow$ classic SGD

$B=N \rightarrow$ GD

The computation of $\mathbf{g}$ can be **parallelized** easily. This is how current deep-learning applications utilize GPUs (by running over $|B|$ threads in parallel).

Note that in the extreme case $B := [N]$, we obtain (batch) gradient descent, i.e. $\mathbf{g} = \nabla \mathcal{L}$.



SGD for Linear MSE

See Exercise Sheet 2.

Computational cost. For linear MSE, what is the complexity (# operations) of computing the stochastic gradient?

(using only $|B| = 1$ data examples)

cost per step	
SGD	$\ell_n = \frac{1}{2}(y_n - \mathbf{x}_n^T \mathbf{w})^2$ $\nabla \ell_n = -\mathbf{x}_n^T \mathbf{e}_n$
GD	$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \ell_n$ $\nabla \mathcal{L} = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$

$\Theta(D)$ (circled in red)
 $\Theta(N \cdot D)$ (circled in red)
 N times cheaper (written vertically in red)
 (indicated by a red arrow pointing from the GD row to the SGD row)

Variants of SGD

SGD with Momentum

$$\nabla \mathcal{L}_n(\mathbf{w})$$

$$\beta_1 = 0.9$$

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{m}^{(t+1)} := \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \mathbf{g} \quad (\text{momentum term})$$

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{m}^{(t+1)}$$

- momentum from previous gradients (acceleration)

EMA

Adam

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{m}^{(t+1)} := \beta_1 \mathbf{m}^{(t)} + (1 - \beta_1) \mathbf{g} \quad (\text{momentum term})$$

$$\mathbf{v}_i^{(t+1)} := \beta_2 \mathbf{v}_i^{(t)} + (1 - \beta_2) (\mathbf{g}_i)^2 \quad \forall i \quad (\text{2nd-order statistics})$$

$$\mathbf{w}_i^{(t+1)} := \mathbf{w}_i^{(t)} - \frac{\gamma}{\sqrt{\mathbf{v}_i^{(t+1)}}} \mathbf{m}_i^{(t+1)} \quad \forall i$$

- faster forgetting of older weights
- is a momentum variant of Adagrad
- coordinate-wise adjusted learning rate
- strong performance in practice, e.g. for self-attention networks

$$\frac{\mathbf{m}_i}{\sqrt{\mathbf{v}_i}} = \frac{\mathbf{g}_i}{\sqrt{\mathbf{g}_i^2}} = \frac{\mathbf{g}_i}{|\mathbf{g}_i|} = \text{sign}(\mathbf{g}_i)$$

SignSGD

pick a stochastic gradient $\mathbf{g}$

$$\mathbf{w}_i^{(t+1)} := \mathbf{w}_i^{(t)} - \gamma \text{sign}(\mathbf{g}_i) \quad \forall i$$

- only use the sign (one bit) of each gradient entry $\rightarrow$ communication efficient for distributed training
- convergence issues

special case of Ada

$$\beta_1 \rightarrow 0$$

$$\beta_2 \rightarrow 0$$

Non-differentiable

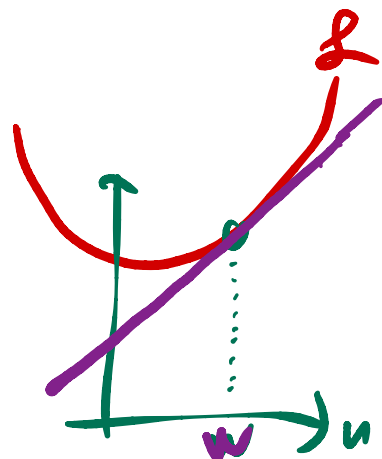
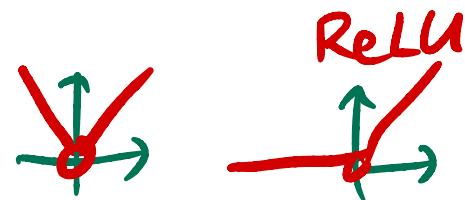
Non-Smooth Optimization

An alternative characterization of *convexity*, for differentiable functions is given by

$$\mathcal{L}(\mathbf{u}) \geq \mathcal{L}(\mathbf{w}) + \boxed{\nabla \mathcal{L}(\mathbf{w})^\top (\mathbf{u} - \mathbf{w})} \quad \forall \mathbf{u}, \mathbf{w}$$

1st-order Taylor approx.

meaning that the function must always lie above its **linearization**.

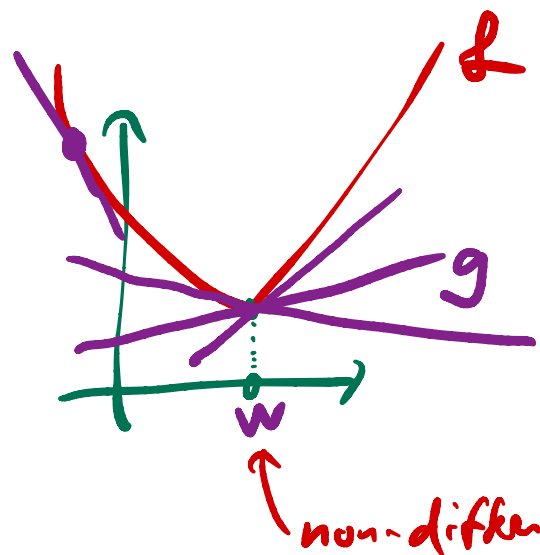


Subgradients

A vector **g** $\in \mathbb{R}^D$ such that

$$\mathcal{L}(\mathbf{u}) \geq \mathcal{L}(\mathbf{w}) + \mathbf{g}^\top (\mathbf{u} - \mathbf{w}) \quad \forall \mathbf{u}$$

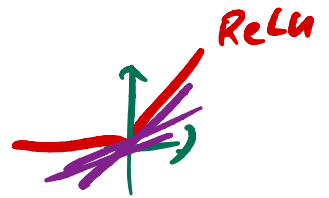
is called a **subgradient** to the function $\mathcal{L}$ at $\mathbf{w}$.



This definition makes sense for objectives $\mathcal{L}$ which are not necessarily differentiable (and not even necessarily convex).

If $\mathcal{L}$ is convex and differentiable at $\mathbf{w}$, then the only subgradient at $\mathbf{w}$ is $\mathbf{g} = \nabla \mathcal{L}(\mathbf{w})$.

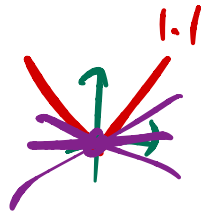
Subgradient Descent



Identical to the gradient descent algorithm, but using a subgradient instead of gradient. Update rule

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma \mathbf{g}$$

← use subgradient



for $\mathbf{g}$ being a subgradient to $\mathcal{L}$ at the current iterate $\mathbf{w}^{(t)}$.

Example: Optimizing Linear MAE

1. Compute a subgradient of the absolute value function

$$\mathbf{g} = \begin{cases} -1 & \text{if } e < 0 \\ [-1, 1] & \text{if } e = 0 \\ 1 & \text{if } e > 0 \end{cases}$$

$$h : \mathbb{R} \rightarrow \mathbb{R}, h(e) := |e|.$$

2. Recall the definition of the mean absolute error:

$$\mathcal{L}(\mathbf{w}) = \text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)|$$

For linear regression, its (sub)gradient is easy to compute using the chain rule. Compute it!

$$\mathcal{L}(\mathbf{w}) = h(q(\mathbf{w}))$$

h : non-differentiable
 q : differentiable

See Exercise Sheet 2.

"subgradient of $\mathcal{L}$ at $\mathbf{w}$ "

$$\mathbf{g} \in \partial h(q(\mathbf{w})) \cdot \nabla q(\mathbf{w})$$

∂h = set of all subgradients of h

Stochastic Subgradient Descent

$$\mathcal{L}(w) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n(w)$$

Stochastic SubGradient Descent
(still abbreviated SGD commonly).

Same, $\mathbf{g}$ being a subgradient to the randomly selected $\mathcal{L}_n$ at the current iterate $\mathbf{w}^{(t)}$.

Exercise: Compute the SGD update for linear MAE.

Linear model training	Smooth MSE	non-smooth MAE
	gradient of $\mathcal{L}$ $\mathcal{O}(N \cdot D)$	subgradient of $\mathcal{L}$ $\mathcal{O}(N \cdot D)$
(sub) GD		
(sub) SGD	gradient of $\mathcal{L}_n$ $\mathcal{O}(D)$	subgradient of $\mathcal{L}_n$ $\mathcal{O}(D)$

per iteration

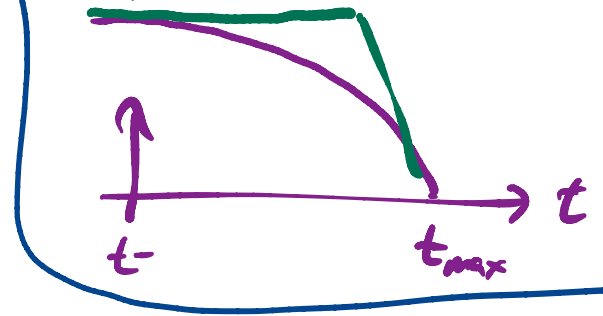
$$\text{theory: } \frac{\text{const}}{t} \dots \frac{\text{const}}{\sqrt{t}}$$

Implementation Issues

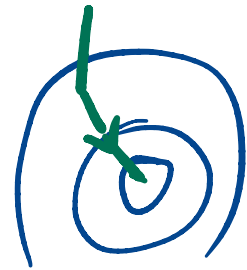
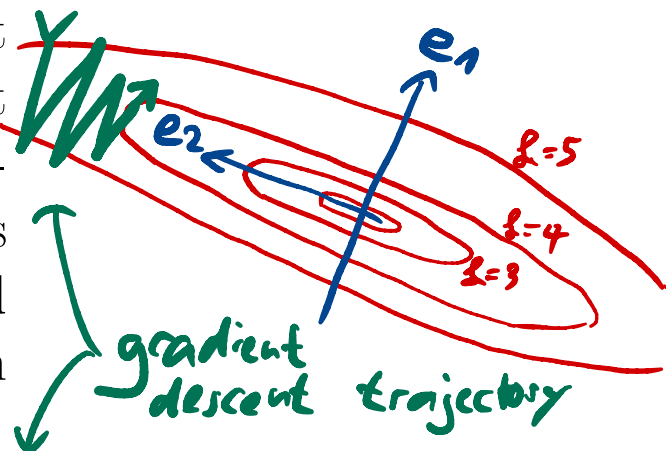
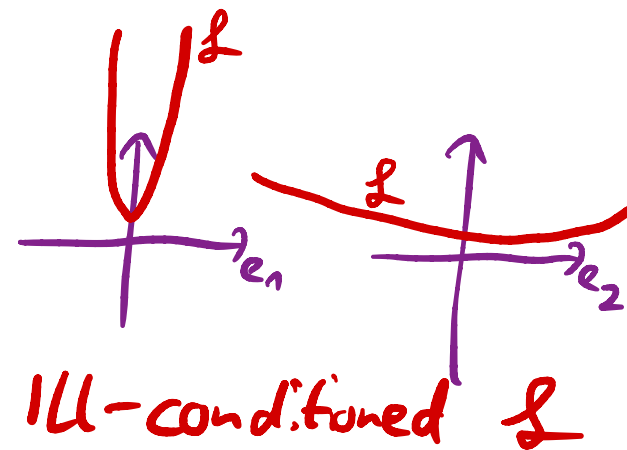
Step-size selection: If γ is too big, the method might diverge. If it is too small, convergence is slow. Convergence to a local minimum is guaranteed only when the step-size γ follows a schedule becoming small enough, where 'small enough' depends on the problem (landscape) parameters.

Feature normalization and pre-conditioning: Gradient descent is very sensitive to ill-conditioning, that is the phenomenon when some coordinates (directions) have change rates vastly different from others.

in practice



Therefore, it is typically advised to normalize input features if possible, and also normalize each layer in a neural network (we will get to this in the later chapters on neural networks). Without this, step-size selection is much more difficult since different "directions" might converge (or fail to do so) at different speed. Rescaling coordinates in this sense is also known as pre-conditioning the optimization problem.

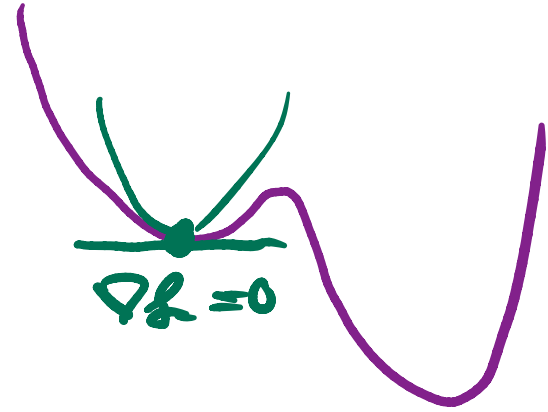


well-conditioned L

Stopping criteria: When $\|\nabla \mathcal{L}(\mathbf{w})\| \rightarrow 0$ is (close to) zero, we are (often) close to the optimum value.

Optimality Conditions

1st For a *smooth* optimization problem, the first-order necessary condition says that at an optimum the gradient is equal to zero. Points of zero gradient are called critical points.



$$\nabla \mathcal{L}(\mathbf{w}^*) = \mathbf{0}$$

2nd We can use the second derivative to study if a candidate point is a local minimum (not a local maximum or saddle-point) using the Hessian matrix,

$$\nabla^2 \mathcal{L}(\mathbf{w}) := \frac{\partial^2 \mathcal{L}}{\partial \mathbf{w} \partial \mathbf{w}^\top}(\mathbf{w})$$

The second-order sufficient condition states that if

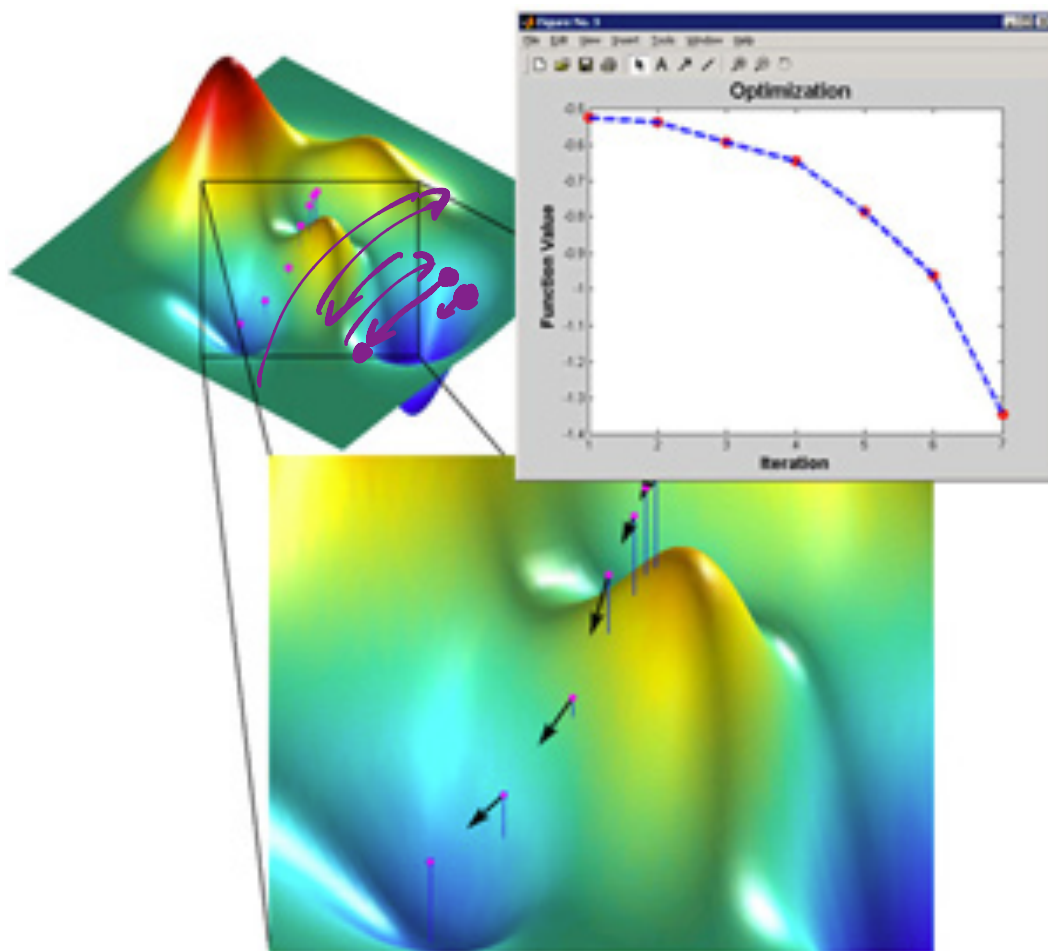
- $\nabla \mathcal{L}(\mathbf{w}) = \mathbf{0}$ (critical point)
- and $\nabla^2 \mathcal{L}(\mathbf{w}) \succ 0$ (positive definite),

then $\mathbf{w}$ is a local minimum.

$\mathcal{L}$ convex
+ $\nabla \mathcal{L} = 0$
⇒ global optimum

The Hessian is also related to the convexity of a function: a twice-differentiable function is convex if and only if the Hessian is positive semi-definite at all points.

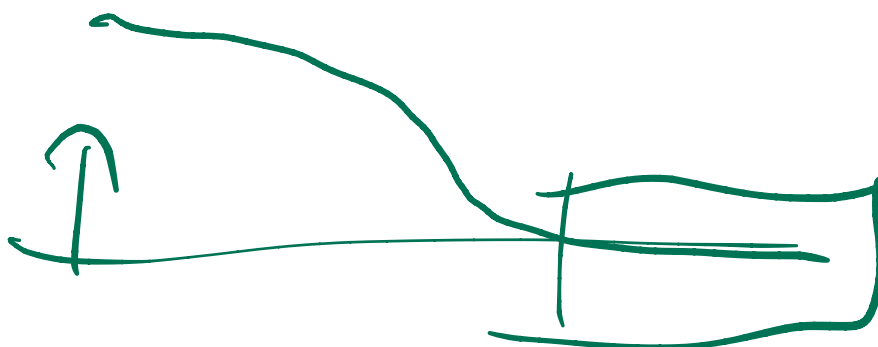
Non-Convex Optimization



*image from mathworks.com

Real-world problems are **not convex**!

All we have learnt on algorithm design and performance of convex algorithms still helps us in the non-convex world.

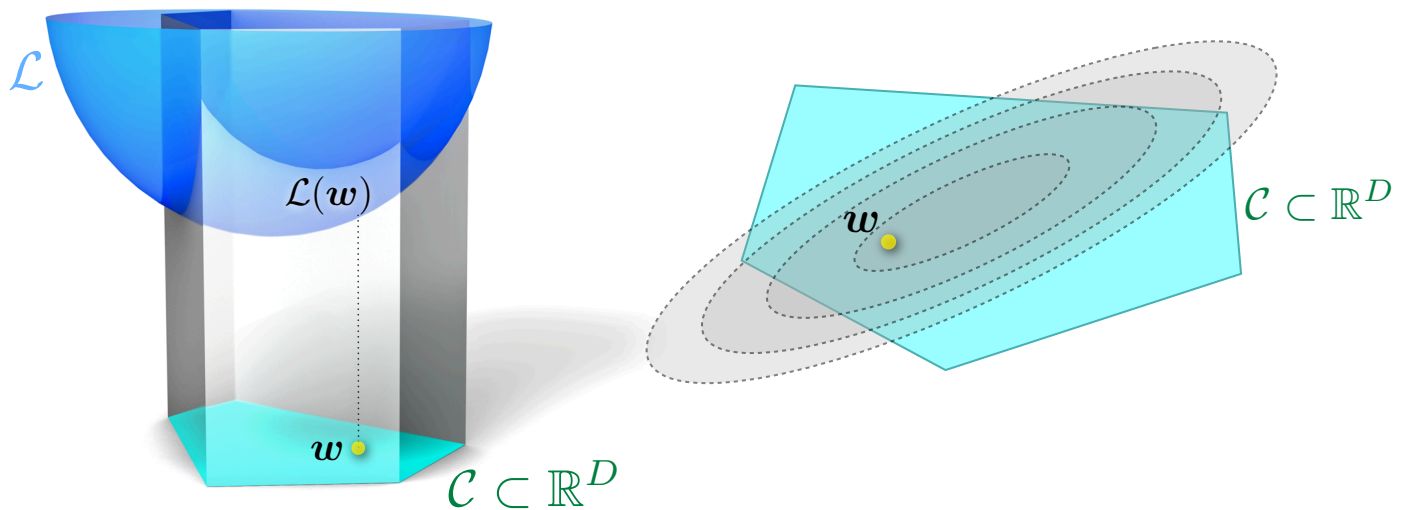


Constrained Optimization

Sometimes, optimization problems come posed with additional constraints:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}), \quad \text{subject to } \mathbf{w} \in \mathcal{C}.$$

The set $\mathcal{C} \subset \mathbb{R}^D$ is called the [constraint set](#).



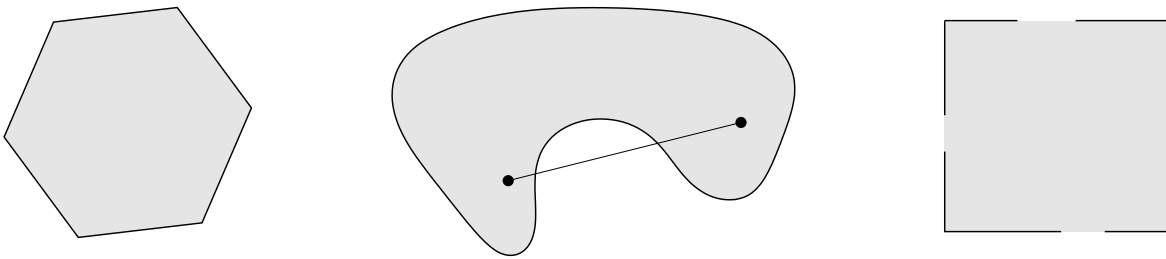
Solving Constrained Optimization Problems

- A) Projected Gradient Descent
- B) Transform it into an *unconstrained* problem

Convex Sets

A set $\mathcal{C}$ is **convex** *iff* the line segment between any two points of $\mathcal{C}$ lies in $\mathcal{C}$, i.e., if for any $\mathbf{u}, \mathbf{v} \in \mathcal{C}$ and any θ with $0 \leq \theta \leq 1$, we have

$$\theta \mathbf{u} + (1 - \theta) \mathbf{v} \in \mathcal{C}.$$



*Figure 2.2 from S. Boyd, L. Vandenberghe

Properties of Convex Sets

- Intersections of convex sets are convex
- Projections onto convex sets are *unique*.
(and often efficient to compute)

Formal definition:

$$P_{\mathcal{C}}(\mathbf{w}') := \arg \min_{\mathbf{v} \in \mathcal{C}} \|\mathbf{v} - \mathbf{w}'\|.$$

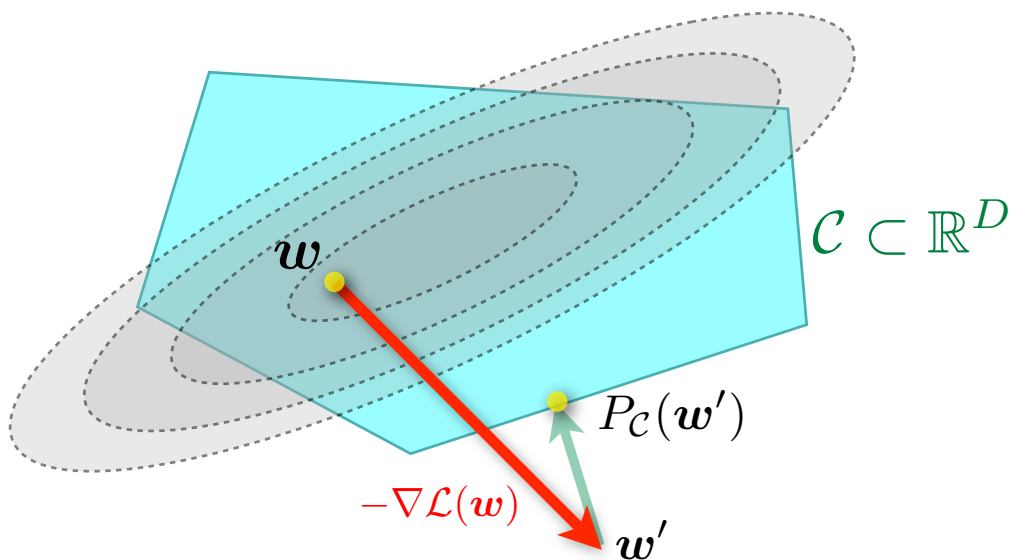
Projected Gradient Descent

Idea: add a **projection onto $\mathcal{C}$** after every step:

$$P_{\mathcal{C}}(\mathbf{w}') := \arg \min_{\mathbf{v} \in \mathcal{C}} \|\mathbf{v} - \mathbf{w}'\| .$$

Update rule:

$$\mathbf{w}^{(t+1)} := P_{\mathcal{C}}[\mathbf{w}^{(t)} - \gamma \nabla \mathcal{L}(\mathbf{w}^{(t)})] .$$



Projected SGD. Same SGD step, followed by the projection step, as above. Same convergence properties.

Computational cost of projection?
Crucial!

Turning Constrained into Unconstrained Problems

(Alternatives to projected gradient methods)

Use **penalty functions** instead of directly solving $\min_{\mathbf{w} \in \mathcal{C}} \mathcal{L}(\mathbf{w})$.

- “brick wall” (indicator function)

$$I_{\mathcal{C}}(\mathbf{w}) := \begin{cases} 0 & \mathbf{w} \in \mathcal{C} \\ \infty & \mathbf{w} \notin \mathcal{C} \end{cases}$$

$$\Rightarrow \min_{\mathbf{w} \in \mathbb{R}^D} \mathcal{L}(\mathbf{w}) + I_{\mathcal{C}}(\mathbf{w})$$

(disadvantage: non-continuous objective)

- Penalize error. *Example:*

$$\mathcal{C} = \{\mathbf{w} \in \mathbb{R}^D \mid A\mathbf{w} = \mathbf{b}\}$$

$$\Rightarrow \min_{\mathbf{w} \in \mathbb{R}^D} \mathcal{L}(\mathbf{w}) + \lambda \|A\mathbf{w} - \mathbf{b}\|^2$$

- Linearized Penalty Functions
(see Lagrange Multipliers)

Additional Notes

Grid Search and Hyper-Parameter Optimization

Read more about grid search and other methods for “hyperparameter” setting:

en.wikipedia.org/wiki/Hyperparameter_optimization#Grid_search.

Computational Complexity

The **computation cost** is expressed using the **big- $\mathcal{O}$** notation. Here is a definition taken from Wikipedia. Let f and g be two functions defined on some subset of the real numbers. We write $f(x) = \mathcal{O}(g(x))$ as $x \rightarrow \infty$, if and only if there exists a positive real number c and a real number x_0 such that $|f(x)| \leq c|g(x)|$, $\forall x > x_0$.

Please read and learn more from this page in Wikipedia:

en.wikipedia.org/wiki/Computational_complexity_of_mathematical_operations#Matrix_algebra .

- What is the computational complexity of matrix multiplication?
- What is the computational complexity of matrix-vector multiplication?

SGD Theory

As we have seen above, when N is large, choosing a random training example $(\mathbf{x}_n, y_n)$ and taking an SGD step is advantageous:

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - \gamma^{(t)} \nabla \mathcal{L}_n(\mathbf{w}^{(t)})$$

For convergence, $\gamma^{(t)} \rightarrow 0$ “appropriately”. One such condition called the Robbins-Monroe condition suggests to take $\gamma^{(t)}$ such that:

$$\sum_{t=1}^{\infty} \gamma^{(t)} = \infty, \quad \sum_{t=1}^{\infty} (\gamma^{(t)})^2 < \infty \quad (1)$$

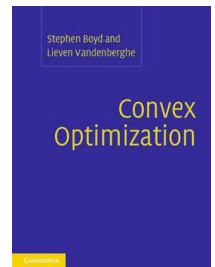
One way to obtain such sequences is $\gamma^{(t)} := 1/(t+1)^r$ where $r \in (0.5, 1)$.

More Optimization Theory

If you want, you can gain a deeper understanding of several optimization methods relevant for machine learning from this survey:

Convex Optimization: Algorithms and Complexity
- by Sébastien Bubeck

And also from the book of Boyd & Vandenberghe
(both are free online PDFs)



Exercises

1. Chain-rule



If it has been a while, familiarize yourself with it again.

2. Revise computational complexity ($\mathcal{O}$ -notation).
3. Derive the computational complexity of grid-search, gradient descent and stochastic gradient descent for linear MSE (# steps and cost per step).
4. Derive the gradients for the linear MSE and MAE cost functions.
5. Implement gradient descent and gain experience in setting the step-size.
6. Implement SGD and gain experience in setting the step-size.